Sodium support: what remains to do

Sodium support is already partly functional in this branch. The safest implementation is to extend the existing shared-renderer integration, repair two confirmed Sodium-specific gaps, and resolve a frame-generation defect that also reproduces without Sodium. Replacing the entire AA pipeline would add risk without evidence that it is needed.

Confirmed finding	Required response
P1 - MIP controls silently miss Sodium terrain	Recognize Sodium terrain shader sources, including OIT variants, and apply the existing bounded bias policy with proof that the compiled pipeline consumes it.
P2 - motion capture disables an optimization	Add an optional bulk vertex-writer adapter that preserves Sodium intrinsics while collecting the same current/previous geometry.
P0 shared blocker - frame generation	Fix the invalid Vulkan feature/SDK presentation path reproduced in the no-Sodium baseline before certifying FG with any renderer.
Coverage gap - reproducible support	Add real Sodium runtime test profiles, version pins and a feature/quality/lifecycle release matrix. API-only compilation does not test its renderer.

What worked in the sampled runs

FXAA, SMAA, MSAA 2x/4x/8x, alpha-to-coverage selection, SSAA 200%/400%, NIS, FSR1, TAA, DLSS Quality/DLAA, FSR2 and FSR3 rendered successfully with Sodium 0.9.2 on Vulkan. Target sizes and native MSAA pipeline metadata matched. Camera, moving-entity and particle inputs were measured; glass/water OIT, reload and menu round trips were exercised.

Meaning of the requested "no bugs" goal
Treat it as concrete release acceptance criteria, not a universal guarantee. These experiments cover one GPU and selected settings. Frame generation failed a baseline validation test; all presets, extreme resolutions, drivers, modpacks and shader loaders remain future qualification work.

Scope: report and implementation plan only. Production mod code was not changed. The normal runClient config/options were preserved. Source, binary and experiment identities are pinned in the appendix.

Versions, scope and evidence levels

Component	Audited identity
Minecraft / loader	26.3 / Fabric Loader 0.19.5; Java 25; Fabric API 0.162.0+26.3
Salt's Anti Aliasing	fabric-26.3 commit 21b0de753ee1; packaged mod 1.1
Primary Sodium target	0.9.2+mc26.3, release tag mc26.3-0.9.2; source 03198cb36ab8
Secondary drift check	0.9.3-beta.1+mc26.3; source d553ceeea0f3; limited smoke matrix
Graphics test system	Windows x64, RTX 3060 Ti 8 GiB, NVIDIA 591.86; driver reports Vulkan 1.4.325; RenderPearl uses the Vulkan 1.2 device contract
Native reconstruction	User-imported DLSS 310.9.1 / Streamline 2.14.1; existing bundled FidelityFX runtime

The current 26.3 Sodium source uses Mojang RenderPearl and has OpenGL and Vulkan draw backends. Advice based on older Sodium OpenGL internals is insufficient here. The installed API dependency already matches 0.9.2, but it is compileOnly: it supplies configuration interfaces, not an installed Sodium renderer.

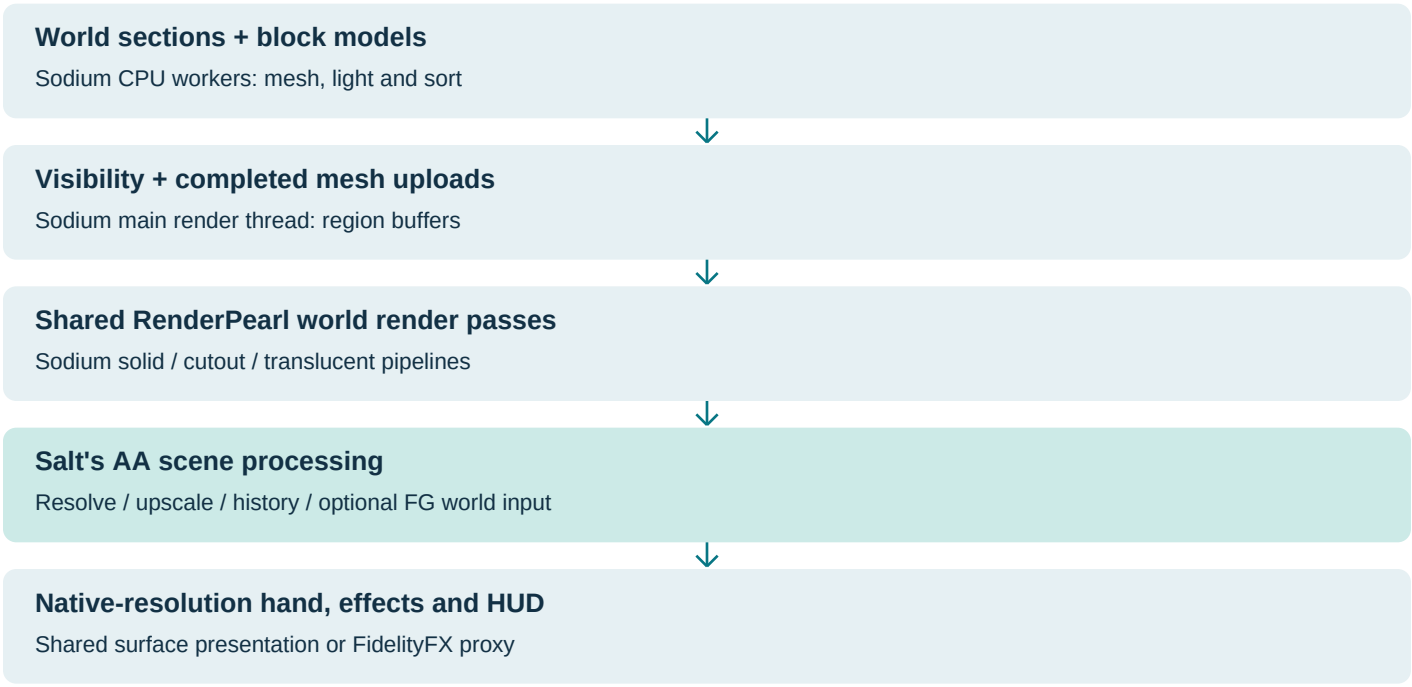
Evidence categories used throughout

Label	Meaning
Measured	Observed in the isolated game instance, pipeline trace, GPU buffer or validator log.
Code-confirmed	Directly established by the pinned source and the mod's committed implementation.
Risk / planned gate	A plausible interaction that needs the stated experiment; not presented as an observed defect.

Sodium and shader-pack support are different projects. No Iris or shader pack was installed in these runs. The September release warning about Iris is historical release context, not proof of its exact October compatibility status. This report certifies neither Iris nor an OpenGL AA backend.

Evidence: [S1-S3] official releases, pinned Sodium checkout and verified runtime metadata; [M1] build.gradle / gradle.properties; analysis.json.

How Sodium renders a frame



Sodium replaces terrain preparation and submission, with additional optimizations for models, particles, textures and culling. It does not create an entirely separate Vulkan device or presentation system in this target. SodiumChunkSection supplies terrain drawing through the RenderPass provided by the shared world frame graph.

This ownership split explains why much of the mod already works: the main-target redirection, world projection hook, frontend pipeline variants and post-world resolve still surround Sodium terrain. The integration should preserve Sodium's mesher, visibility graph, region allocations, batching and upload lifecycle.

Recommended boundaries

Keep scene-target ownership in the mod's controllers. Keep terrain buffers and mesh lifetimes in Sodium. Add optional shader classification and vertex-writer adapters at their narrow boundaries. Keep native upscalers attached to the shared device and command encoder. Preserve hand/HUD ordering after reconstruction.

Evidence: [S4] LevelRendererMixin; [S5] SodiumChunkSection; [S6] SodiumWorldRenderer; [M2] GameRendererMixin / RenderRuntime.

Terrain preparation and GPU submission

ChunkBuilder schedules mesh work on worker threads. Completed results are consumed on the main render thread, where region buffers and render lists are updated. Visibility, fog occlusion and face culling reduce the submitted geometry; translucent geometry has its own sorting and index-management behavior.

Mechanism in 0.9.2	Integration consequence
Compressed terrain vertices: 20-byte stride	Do not decode them as ordinary entity float-position vertices. The bulk object-motion adapter belongs to its actual supplied vertex format.
Region dimensions: 8 x 4 x 8 sections	Keep region-local coordinates and camera-relative translation. Avoid converting large world positions to float too early.
Region-level batches + shared/indexed buffers	Preserve primitive topology, index type, buffer slices and draw preparation. Do not remesh terrain just because AA quality changes.
20-byte push-constant range	Cloned pipelines must retain it, all bind-group layouts and vertex formats. Dropping it would break the terrain draw contract.
EXT multidraw or indexed-indirect Vulkan path	The selected pipeline still passes through the frontend. Both backend paths need eventual qualification; this hardware primarily exercises multidraw.

DefaultChunkRenderer binds the compiled terrain pipeline, u_Globals, section timing, lightmap and block atlas, then draws prepared region batches. Its Vulkan DrawContext retains the provided pass; it does not silently replace the AA-selected native pipeline afterward.

Static terrain camera velocity should continue to come from depth reprojection. Capturing every terrain vertex on the CPU each frame would undermine Sodium's design and is unnecessary for static chunks. Moving models and particles need their separate motion paths.

Evidence: [S7-S10] ChunkBuilder, RenderSectionManager, RenderRegion, CompactChunkVertex; [S11-S13] DefaultChunkRenderer, DrawBackend / DrawContext.

The current integration contract

Boundary	Current behavior / what was checked
Scene ownership	GameRenderer.renderLevel begins redirection before resolution-dependent world state; end-of-world processing resolves it.
Terrain attachment reuse	SodiumChunkSection renders into the provided world pass. Traces showed 853x480 for DLSS Quality and 2560x1440 for 200% SSAA at 720p output.
Projection	Sodium copies the matrix at the same ProjectionMatrixBuffer call used by the jitter hook. Measured matrices agree with the expected jittered projection.
MSAA pipelines	FrontendRenderPass selects immutable variants; native compile metadata sets raster sample count. Sodium terrain traces matched 2x/4x/8x attachments.
OIT	The engine creates depth-bounds, transmittance and accumulation passes. Their float formats and single-sample attachments must stay intact.
World motion	Camera reprojection plus captured object geometry and billboards remains active. The object wrapper currently causes a Sodium optimization fallback.
Shader LOD controls	The selector only recognizes minecraft:core/terrain. Sodium uses a different fragment source, so this feature does not carry over.
Settings	The public Sodium config entrypoint opens the same AA screen and returns to its parent; observed in the game.

Contract tests instead of a broad renderer rewrite

Record the actual color/depth target, dimensions, formats, samples, selected source shader, jitter/projection and reset generation at each boundary. Assert these at runtime in development. A renderer adapter should correct a demonstrated mismatch, not replace working frame-graph hooks speculatively.

Evidence: [M2-M8] GameRendererMixin, FrontendRenderPassMixin, VulkanMsaaPipelineVariants, controllers and VulkanTextureDetail; [S4-S6], [S14].

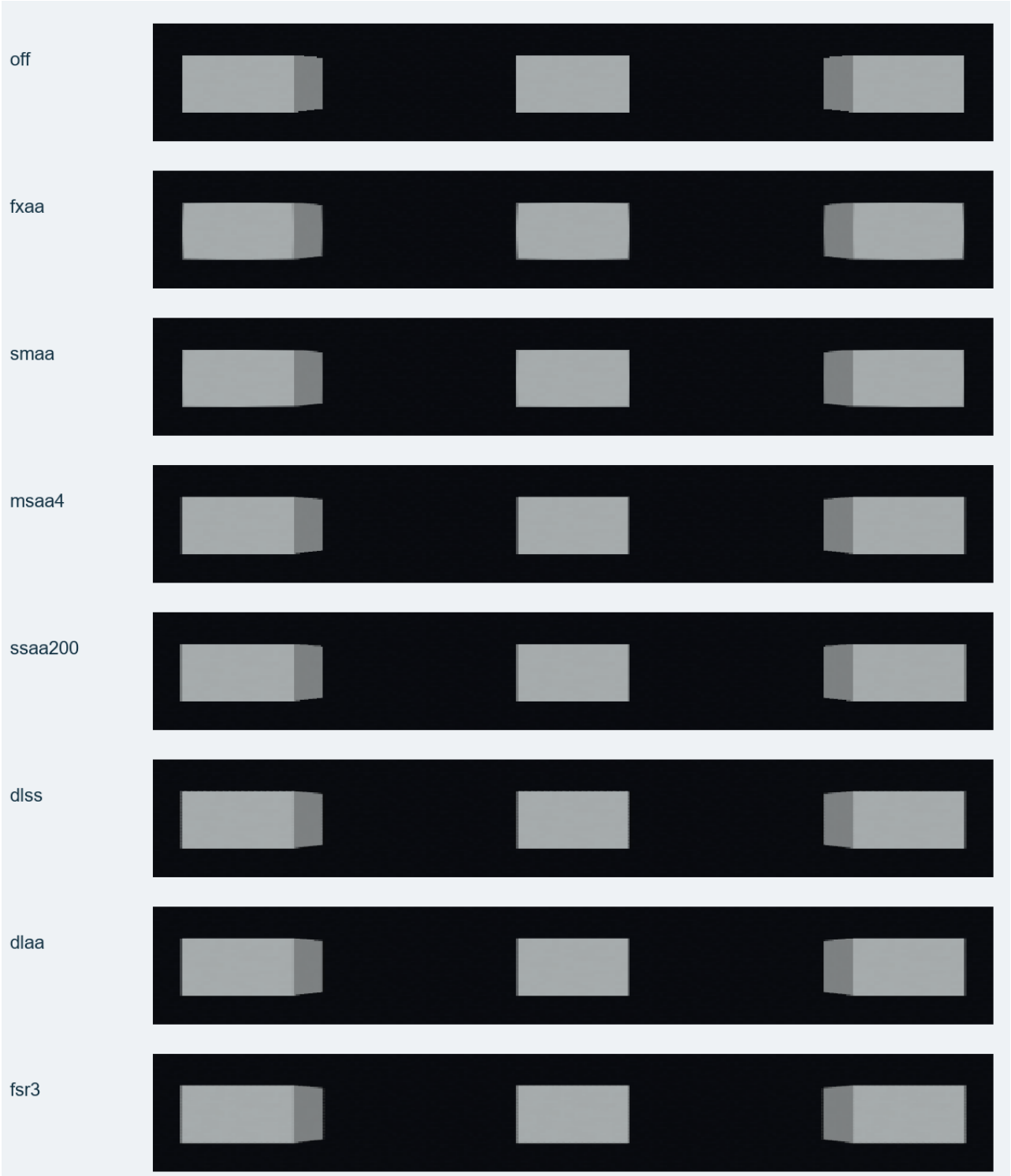
What was tested in game

Path / controls	Sodium 0.9.2 observation
Off / FXAA / SMAA	Rendered; native scene size; representative output crops inspected.
MSAA 2x, 4x, 8x	Rendered; attachment/native pipeline counts agree.
MSAA 4x + alpha coverage	Cutout variant selected; blended terrain retains its blend policy.
SSAA 200%, 400%	Rendered into 2560x1440 and 5120x2880 before 1280x720 resolve.
NIS / FSR1 Quality	Rendered into 986x554; normal spatial resolve path active.
TAA	Rendered; projection/jitter agreement checked.
DLSS Quality / DLAA	853x480 and 1280x720 inputs respectively; healthy native dispatch.
DLSS custom MIP bias -2	Rendering stayed healthy, but Sodium terrain received no bias define.
FSR2 / FSR3 Quality	853x480 reconstruction input; healthy dispatch.
OIT with MSAA / DLSS / FSR3	Glass/water fixture rendered; float OIT passes observed in the beta trace.
Dynamic and lifecycle coverage	Yaw/pitch/strafe, moving pig, particles, floating-point target switch, resource reload and menu round trip.
FSR3 frame generation	No-Sodium validator baseline failed before a successful capture. Sodium FG is not qualified.

The stable matrix used eight captured frames per case after 100 warmup frames at 60 FPS. Dynamic input tests used 24 frames. The beta matrix sampled nine cases. Most static scenes used a frozen simulation; dynamic entity/particle tests unfroze it. Extra sharpening was zero. Core and synchronization validation were enabled.

Limitations: one GPU/driver, mainly 720p output, one map and selected settings; no indirect-only GPU, long play session, exhaustive resource-pack matrix, every quality/model/sharpening preset or shader-loader combination. The complete capture inventory is in analysis.json.

Output comparison: rendering is active



Sodium 0.9.2: enlarged output crops from Off, FXAA, SMAA, MSAA 4x, SSAA 200%, DLSS Quality, DLAA and FSR3. Source crop x=440..839, y=310..419. Nearest-neighbor enlargement preserves pixels.

This mostly axis-aligned fixture confirms visible output and alignment; it cannot rank AA quality or establish freedom from shimmering. A stronger release fixture must include moving diagonals, subpixel rails, fine foliage, alpha textures and variable camera velocity. Compare each renderer against its own native reference, since Sodium can change lighting, meshing and texture sampling independently of AA.

Confirmed gap: terrain MIP controls

Code-confirmed and measured. VulkanTextureDetail.select only accepts a fragment shader identifier equal to minecraft:core/terrain. Sodium's terrain pipeline uses sodium:blocks/block_layer_opaque for solid, cutout and translucent terrain. A request for custom bias -2 was saved correctly, yet the observed Sodium pipelines contained no SALTS_MIP_BIAS define in both audited versions.

Required implementation

Introduce a shader-family classifier that recognizes the pinned Sodium fragment source as well as vanilla. Classify by source shader identity, not just pipeline location: OIT pipelines with Sodium shaders can have minecraft-namespace pipeline names. Build variants from the actual source pipeline and preserve defines, layouts, push constants, formats, topology, blend/depth/write state and vertex bindings.

Reuse the existing bounded bias policy and source wrapper where it remains valid. Sodium samples with both explicit gradients and explicit LODs, including its nearest/RGSS paths; adjust each intended sampling operation once. Keep its alpha cutoff and UV-resampling math intact. Do not overwrite a shader pack's own policy.

Acceptance gate	Required result
Compiled-path proof	Off has no bias; Custom -2 produces the expected define and a matching shader variant for classic and OIT terrain.
Visual / numerical reference	Known minified texture pattern changes its LOD behavior; compare slow movement against Sodium DLAA and native references.
Reload and combination tests	Bias survives pack reload, filtering changes and mode switches. MSAA/format variants compose without duplicating the adjustment.
Unsupported source	Explain the unavailable control or disable it with a reason; never present an active slider whose value is ignored.

Evidence: [M7] VulkanTextureDetail; [M8] TextureDetailShaderPatch; [S15-S17] ShaderChunkRenderer and terrain shaders. Pipeline traces: sodium092-dlss-mip and sodium093-dlss-mip.

Confirmed gap: optimized vertex writing

The current EntityMotionGeometry wrapper implements VertexConsumer but not Sodium's VertexBufferWriter. Sodium's CubeMixin attempts conversion, finds the wrapper unsupported and uses its ordinary vertex path. The game logged that exact wrapper as not supporting optimized vertex writing. Moving geometry was still captured: this is a confirmed optimization loss, not evidence of missing entity vectors.

Required optional adapter

Create a Sodium-aware writer only when that mod/API is available. Support both ordinary VertexConsumer calls and the bulk push(MemoryStack, long, int, VertexFormat) API used by these versions. Resolve position offsets and stride from the supplied format and validate the encoding; do not use obsolete API examples built around a different format-description type.

Copy needed current positions before forwarding push. Sodium documents that source memory is undefined after that call, and its stack-backed data cannot be retained. Delegate exactly once, preserve canUseIntrinsics behavior, avoid duplicate motion capture, and keep the existing budget, topology, identity and camera-relative matching rules. Unknown formats should use a safe reported fallback.

Correctness	Performance
Compare bulk and scalar captured positions for cubes, items, tint, overlay, pose transforms and camera motion.	Use cached format decoders and reusable position storage. Avoid per-vertex objects and repeated full-buffer conversions.
Test mixed wrappers, births, cap overflow, topology changes and Sodium absent.	Measure CPU cost in a dense moving-entity scene, with equal visibility and settings. Report overhead separately from GPU reconstruction time.

No quantitative FPS loss is claimed from the warning alone. Frame limiting and Vulkan validation would distort such a comparison. The implementation must prove that restoring the fast path does not accidentally bypass motion capture.

Evidence: [M9] EntityMotionGeometry; [S18-S20] CubeMixin, VertexConsumerUtils / Tracker and public VertexBufferWriter API.

FXAA, SMAA and sharpening

These algorithms consume the final world image rather than Sodium's compressed terrain buffers. The existing post-world hook is therefore the correct integration point. Healthy output was captured with Sodium for FXAA and SMAA; no separate Sodium terrain implementation is justified by these tests.

Controls / behavior	Implementation and release checks
FXAA subpixel, edge threshold, minimum contrast	Keep resolution uniforms tied to the actual post-effect target. Test their extremes on diagonal edges and fine texture detail.
SMAA preset and edge threshold	Preserve edge/weight/neighborhood ordering, lookup textures and intermediate dimensions. Confirm no extra MSAA variant is forced onto single-sample post targets.
Sharpener type, strength and limit	Apply once at the intended output resolution. Compare at matched strength; avoid confusing sharpening halos with reconstruction errors.
World-only placement	Resolve before native hand/HUD composition. Verify inventory/chat/F3/text and first-person items remain sharp and correctly layered.
Resource reload / filtering	Shader sources and cached pipelines must refresh without stale defines or persistent sampler changes.

What remains before declaring complete support

Add a native Sodium reference set containing diagonals, fences, foliage, signs, emissive pixels and animated textures. Capture still and moving cameras at all quality presets and slider limits. Test zero-strength behavior and reset/persistence. The present brief output crops establish execution, not a complete spatial-AA quality audit.

Avoid inserting these effects inside Sodium's region draw loop: that would process incomplete terrain layers and risk applying the effect many times. Keep one scene resolve at the existing frame boundary.

Evidence: [M2-M3] GameRendererMixin / VulkanScenePostProcessor; current FXAA/SMAA shader resources; captured sodium092-fxaa and sodium092-smaa outputs.

MSAA and alpha-to-coverage

Measured: Sodium solid/cutout/translucent draws selected 2x/4x/8x variants matching the active multisampled scene attachments. The 4x coverage case selected a cutout coverage variant. Keep these shared frontend/native compile hooks rather than adding a second Sodium-specific sample-count system.

Boundary	Required invariant
Solid/cutout scene pass	All attached color/depth resources and the selected native graphics pipeline use matching sample counts.
Pipeline cloning	Retain Sodium's compressed vertex format, 20-byte push constants, bind groups, depth/cull/topology and existing shader defines.
Alpha policy	Only eligible cutout materials use coverage. Classic alpha-blended materials keep blending; preserve Sodium's alpha threshold.
OIT	Depth bounds, transmittance and accumulation retain their own float formats and single-sample scope. Use the resolved depth required by the engine.
Resolve / depth	Color resolves once; depth used for transparency and later passes must represent compatible samples and dimensions.
Hardware limits	Use supported format/sample masks, expose the effective sample count, and avoid labeling a clamped or failed path as the requested quality.

The observed beta OIT terrain pipelines were minecraft:pipeline/oit_depth_bounds_sodium_terrain with RGBA32F and transmittance/accumulation variants with RGBA16F, all single-sampled. Their fragment sources remain Sodium shaders. Pipeline-name namespace alone is not a safe feature detector.

Remaining gates include cutout foliage at distance, intersecting translucent surfaces, underwater transitions, both Vulkan draw backends, every supported sample count, and native hand AA. Passing an opaque scene alone is insufficient.

Evidence: [M4-M6] FrontendRenderPassMixin, VulkanMsaaPipelineVariants and VulkanSceneMsaaController; [S11], [S15]; OIT pipeline traces.

SSAA, NIS and FSR1

Sodium terrain followed the redirected render target. At 720p output, 200% and 400% SSAA used 2560x1440 and 5120x2880 scene targets. NIS/FSR1 Quality used 986x554. This validates the basic target boundary; it does not prove every extreme scale or optional filter configuration.

Feature	Required work / qualification
SSAA scale + area/tent filtering	Keep world viewport/global dimensions at the internal size and output resolve at display size. Validate filter footprints, linear-light option and odd dimensions.
SSAA capacity	Account for format, samples, output size and VRAM. Existing maximum-texture-size checks are useful; make the unavailable/effective scale clear instead of promising 800% everywhere.
NIS / FSR1 quality	Query/use each mode's intended internal size. Keep pre-AA and output sharpening in their intended order, and avoid duplicate sharpening.
Texture LOD	Extend the MIP selector to Sodium. Test nearest, linear, RGSS, mip levels and resource-pack texture resolutions together.
Visibility / aspect	Retain stable culling. Resizing targets should not trigger a complete terrain rebuild unless a genuine renderer resource contract requires one.

Texture detail is a quality tradeoff

Sodium's shader explicitly computes gradients, nearest-style UV adjustments and an RGSS LOD. A bias patch must respect those calculations. Negative bias may recover detail and also increase temporal shimmer. Keep user control, compare against native Sodium references, and avoid applying terrain bias blindly to entities or fonts without mip chains.

Test window sizes around rounding boundaries, display resize, render-distance changes and low-memory behavior. Distinguish a rejected resource allocation from a Sodium rendering incompatibility.

Evidence: [M7-M8], [M10] VulkanSceneScaleController; [S16-S17] Sodium terrain sampling; captured SSAA/NIS/FSR1 pipeline dimensions.

TAA and temporal camera agreement

Sodium stores the world projection from ProjectionMatrixBuffer, and its uniform manager uploads that matrix with the model-view transform. In the installed mixin combination, the copied matrix already includes the mod's jitter. Do not add another jitter offset or reverse motion-vector Y based on assumptions about older renderers.

Camera experiment	Samples	p95 error, input pixels
Yaw	125,747	0.0010809
Pitch	128,050	0.0004697
Strafe	125,990	0.0006835

The 24-frame GPU readbacks reconstruct a static point from captured jittered depth, project it through the previous/current unjittered matrices, and compare with RG16F velocity. Sodium's projection differed from the calculated jittered projection by at most about 1e-10 in the recorded float values.

TAA-specific qualification

Verify history weight, luma/depth rejection, variance clipping, jitter phases and history filters across camera acceleration, view bob, hurt/nausea, FOV changes and perspective switches. Check entity/particle coverage, culling reappearance, section rebuilds, dimension changes, resource reload and resumed rendering. Add an executable matrix-agreement regression test to guard future mixin-order changes.

Sodium chunk fade changes fogged color while geometry may be stationary. New mesh visibility and visible-only animation also create disocclusions or color changes. These are validation risks, not defects demonstrated by the camera tests. Avoid resetting global history every time a section uploads; that could produce continual instability during streaming.

Evidence: [S4], [S14], [S21] GameRendererMixin, UniformBufferManager and chunk fade shader; [M11] VulkanSceneTemporalController; s92-camera-* GPU inputs.

DLSS Super Resolution and DLAA

Quality and native DLAA dispatched successfully with Sodium. The tested camera convention, RG16F velocity, reverse-Z depth and scene-target sizes remained valid. Moving-pig geometry and billboard particles reached the input path. The separate native-resolution hand/HUD architecture should remain in place.

Input / behavior	Requirement for complete Sodium support
Color + depth + velocity + jitter	All describe the same frame, world projection, viewport and pixel convention. Assert this after every target/quality change.
Object motion	Preserve the measured geometry capture while adding the optimized writer adapter. Test animated models, items, moving blocks, caps and new topology.
Particle layers	Keep alpha/depth dominance policy and exact previous/current billboard transforms. One vector per pixel cannot encode all moving translucent layers.
Color encoding	Vanilla Sodium is the audited SDR contract. Floating-point target plumbing worked, but forcing the flag does not turn gamma content into HDR or certify a shader pack.
Quality / DLAA / model requests	Test each quality and K/M/L/Auto request with matching native sizes, reset semantics and source diagnostics. Driver overrides can supersede a request.
History controls	Do not reintroduce unsupported legacy bias-current-color tags as a modern-model workaround for Sodium chunk fade.

The highest-value rendering change here is the terrain MIP adapter. A separate DLSS implementation inside Sodium's chunk renderer would duplicate existing device/evaluate work and make world/hand/UI ordering harder to maintain.

No DLSS frame generation, multi-frame generation, ray reconstruction or arbitrary shader-pack support is implied by these SR/DLAA tests. Those are distinct features and contracts.

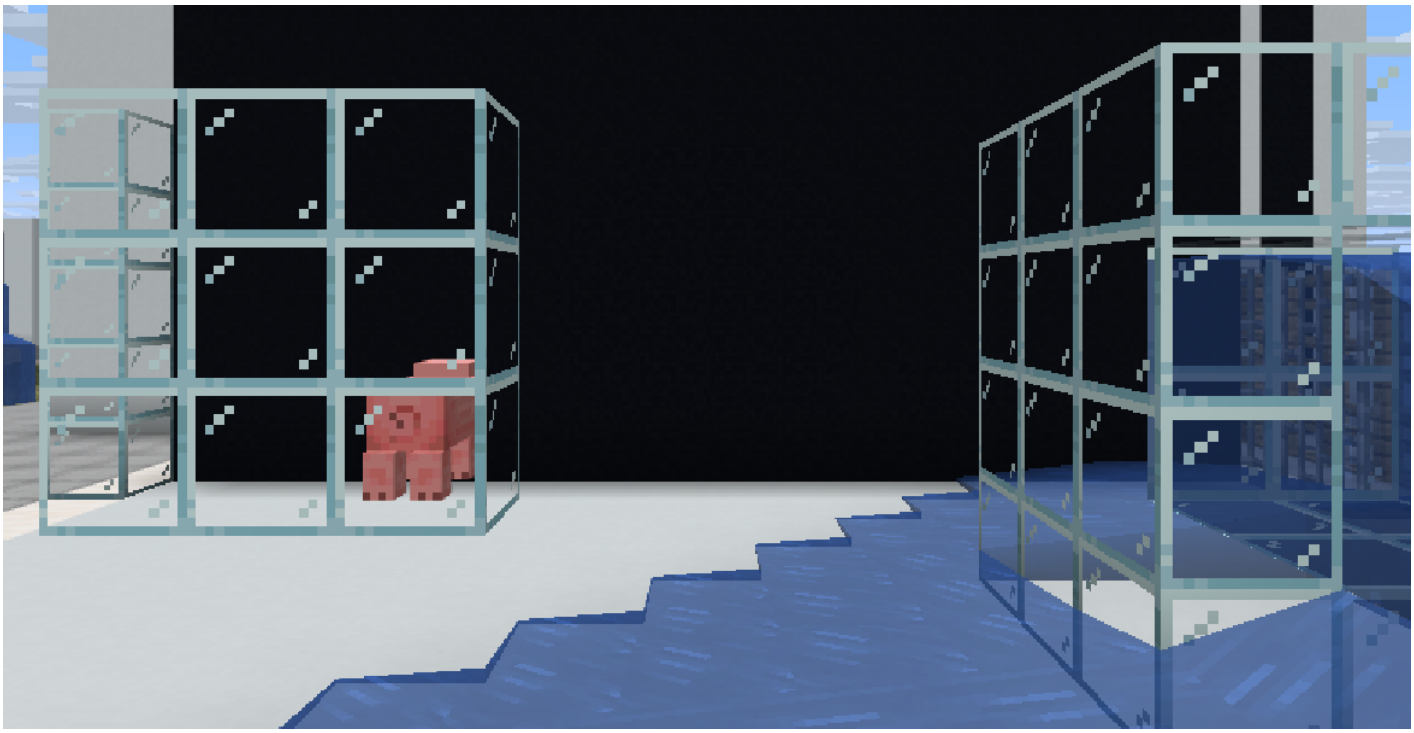
Evidence: [M9], [M11-M12] geometry, temporal and DLSS controllers/bridge; [S14-S17]; s92-entity-motion, s92-particles and Sodium Quality/DLAA captures.

FSR2 and FSR3 reconstruction

Both methods rendered with Sodium at 853x480 input for 1280x720 Quality output. They rely on the shared source/depth/motion path, the existing linear-color transfer and the mod's reactive/composition handling. Their integration should continue to run after complete world color is available.

Classic transparency and improved transparency

The mod captures opaque color before classic transparency by closing the solid pass and continuing with a new translucent pass. The OIT path captures before executeOit. Sodium substitutes terrain draws through these passes, so the boundaries remain relevant. Validate both routes with actual visible transparent terrain, not merely the option enabled in an opaque fixture.



Sodium glass/water fixture with improved transparency and MSAA. Enlarged screenshot confirms the exercised layer overlap; it is not a quantitative transparency-quality certification.

Preserve FSR's own supported masks; their contract differs from modern DLSS. Compare opaque versus full scene capture for moving glass, water, particles and emissive/animated surfaces. Ensure color/depth copy ordering is legal, mask dimensions match input size, and MSAA depth is resolved before single-sample sampling.

Test all quality ratios and native sharpness, the reactive/composition sliders, sudden lighting changes, disocclusions, low input resolution and odd target dimensions. Changes in Sodium texture filtering, fluid shaping or culling must not silently invalidate the chosen mask assumptions.

Evidence: [M13] LevelRendererMixin / VulkanSceneFsrController / temporal mask renderer; [S5], [S15]; classic/OIT fixture captures and zero-error Sodium validator logs.

Frame generation: a shared release blocker

The no-Sodium validation run failed when entering FSR3 frame generation, before a successful FG capture. The first reported shader issue was StorageImageReadWithoutFormat without the required enabled core feature (or the stated alternative capability). The device setup currently enables unformatted writes for DLSS, but does not enable this required read feature for the FG path.

Observed evidence	Interpretation
Shader-module VUID 08740 in native FG swapchain creation	A confirmed Vulkan capability declaration mismatch in the baseline path.
NULL pipeline / incompatible dispatch state, then presentation failure	Later errors may cascade from initialization failure; do not treat every message as an independent root cause.
Native graphics-pipeline lineWidth=0 messages	Investigate the exact FidelityFX pipeline creation path and runtime version after correcting the first capability error.
Semaphore reuse / unwaited acquire messages	Audit presentation ownership and synchronization after restoring valid initialization. Keep separate per-image and in-flight lifetimes.
Same baseline has no Sodium installed	No evidence supports calling this a Sodium-specific FG failure. Sodium FG remains unqualified.

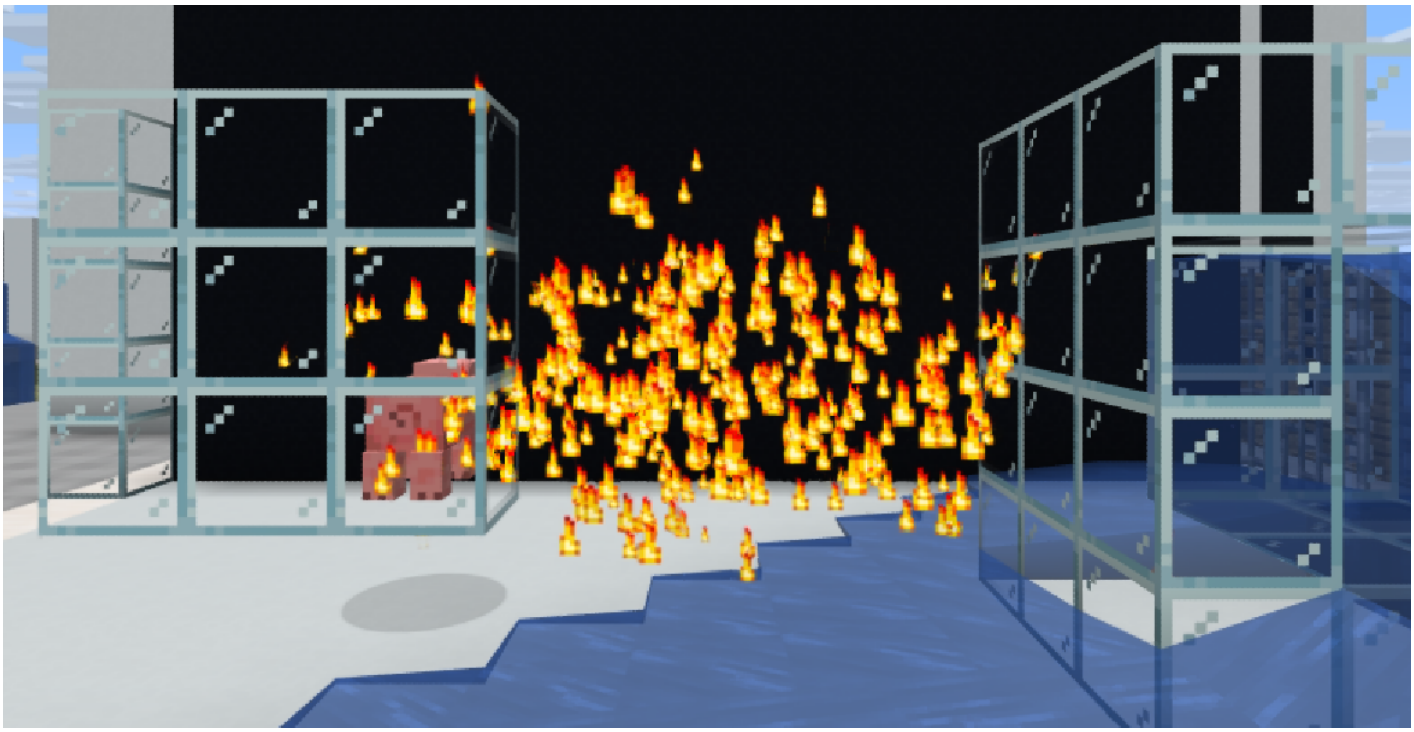
Required implementation sequence

Query the full native feature requirements before device creation, enable only supported features, and refuse FG with a clear reason when they are unavailable. Treat native swapchain creation as transactional: do not report active FG if essential pipelines failed. Rebuild/update or narrowly repair the SDK path if invalid pipeline state persists; do not globally suppress validation.

Then validate swapchain acquire/present ownership, native callbacks, image layouts, fences/semaphores, resize, minimize, Alt-Tab and direct exit. Finally repeat with Sodium: stable and beta, classical/OIT transparency, moving camera/hand, raw versus displayed FPS and low/high base frame rates. Capture actual generated frames and pacing, not just an increased FPS label.

Evidence: [M14] VulkanBackendMixin, VulkanGpuSurfaceMixin, FsrRuntime and native FSR bridge; launch-vanilla.log. The later successful Sodium runs deliberately exclude FG.

Entity and particle motion with Sodium



Sodium + DLSS particle scene, enlarged without smoothing. The fixture includes glass and moving water; raw velocity was also read from the GPU.

24-frame capture	Measured result
Moving pig	252 submitted triangle vertices per frame in the object-motion draw.
Flame billboards	277 to 313 matched quads; 6,683 to 7,837 nonzero velocity pixels in the normalized particle ROI.
Flames + OIT	262 to 304 matched quads; 6,706 to 8,036 nonzero ROI pixels.

The current scalar fallback still collects entity geometry correctly. Sodium's particle optimization occurs during quad vertex emission; the mod captures earlier extraction data, so its billboard position/rotation/scale history is preserved. This is a useful existing boundary to retain.

Release gates still need model variants, dropped items, moving blocks, block entities, nameplates/special effects, capped geometry, camera-relative translation and reappearance after culling. Stable identity/topology is a real requirement. Coverage counters describe intercepted renderer families, not proof of universal support for every modded entity.

Evidence: [S18-S20], [S22] QuadParticleRenderStateMixin; [M9], particle capture / motion renderer; analysis.json. ROI x=0.31..0.70, y=0.38..0.62; nonzero threshold 0.001 input pixels.

Settings, versions and development profiles



Opened through Sodium's external AA page. Done returned to Sodium in the live UI. The public config API integration already works in the tested stable version.

Keep the existing ConfigEntryPoint/external-page link and the shared config model. Do not maintain a second set of AA values inside Sodium. Qualify Apply/Done/Escape behavior with pending Sodium changes, GUI scaling, search, reopen and reset/persistence. The beta contains an upstream onClose fix; include that path in the version matrix.

Add an opt-in development/test profile that installs the full pinned Sodium mod. The current compileOnly sodium-fabric-api dependency is useful for building the menu integration but cannot establish renderer compatibility. Keep a no-Sodium runtime profile too, to catch accidental class-loading dependencies. Lock version/checksum and keep the public JAR from bundling Sodium or private NVIDIA runtime files.

The supported renderer target should explicitly be Sodium + Vulkan. On OpenGL the current mod disables its AA backend. If parity on OpenGL is desired, it is a separate backend/native interop project, not a small Sodium menu fix. RTX/native-runtime/OS capability limits also remain feature-specific.

Evidence: [M1], [M15] fabric.mod.json / SaltsAntiAliasingSodiumConfig / config screen; [S23] ConfigManager and public config API; [S2] beta release notes.

Implementation plan and dependencies

Phase	Concrete deliverable	Pass gate
0 - reproducibility	Pin stable/beta profiles, record renderer/backend and native metadata; reproduce the baseline FG failure separately.	A no-Sodium control and real Sodium runtime run use the same mod artifact and isolated settings.
1 - MIP adapter	Extend shader-family recognition in VulkanTextureDetail; add Sodium sampling tests and immutable variants preserving classic/OIT state.	Requested bias reaches the compiled Sodium shader and affects a known minification fixture once.
2 - motion fast path	Add optional Sodium VertexBufferWriter adapter around object capture, with safe bulk position extraction and fallback.	Bulk/scalar geometry matches; no optimized-writer warning; measured CPU overhead is bounded.
3 - FG correctness	Complete required feature negotiation, transactional SDK initialization and native presentation/sync repairs.	No-Sodium FG passes Vulkan validation and generated-frame/pacing checks before Sodium testing.
4 - lifecycle / coverage	Assert target/matrix contracts, reset on real discontinuities, test visibility/rebuild/color changes and clean resource retirement.	All feature families pass the declared matrix, including reload, resize, unload, failure/retry and direct exit.
5 - release policy	Document supported versions/backends/hardware and expose effective/unavailable controls honestly.	Published support claims match completed tests; unsupported combinations explain their limits.

Suggested adapter boundaries: a small optional Sodium shader-family adapter and a bulk object-motion writer. Existing scene controllers, target redirection, frontend pipeline variants and native reconstruction remain authoritative. An internal scene-input contract can formalize dimensions/formats/samples/matrices and reset generation without forcing a renderer rewrite.

These are proposed changes, not implementations already present in this report. MIP support and motion intrinsics are the demonstrated Sodium-specific implementation work; other rows are release gates or the shared FG repair.

Phase numbers group ownership rather than permit a partial FG release: the shared P0 blocks certification from the outset and can be repaired in parallel with the Sodium adapters.

Acceptance matrix: image and data correctness

Test family	Required coverage / objective pass condition
Mode and parameter coverage	Every listed mode; every quality preset; DLAA; K/M/L/Auto; shader thresholds, history filters, jitter phases, AA sample/scale choices, pre-AA, sharpeners, MIP and mask controls.
Render contract	Actual color/depth formats, extent, samples and selected pipeline agree. No missing source, invalid layouts, unsupported shader capabilities or silent linear-resolve fallback.
Projection and velocity	One jitter application; vectors exclude jitter; compare GPU vectors with CPU ground truth. Suggested p95 static-geometry error below 0.01 input pixels in the controlled fixture.
Geometry and transparency	Animated mobs, held/dropped items, moving blocks, foliage, intersecting glass/water, alpha particles and depth occlusion. Inspect disocclusion, ghosting, halos and floating edges at enlarged scale.
Detail / sampling	Native renderer references, slow camera motion, thin diagonals/rails, distant leaves, text and 16/64/high-resolution texture packs; filtering/RGSS/anisotropy/mip combinations.
Composition	Native hand, screen overlays, HUD, F3/FPS, chat and menus remain correctly placed and sized. Frame generation protects UI/hand and tracks moving world inputs.
Limits	Odd sizes and aspect ratios; 720p/1080p/1440p/4K; high SSAA/MSAA; low memory; out-of-range formats/samples and unavailable native runtimes.

Image gates should use repeatable scene/camera paths and annotated error regions. A single whole-image average can hide thin-edge artifacts. Use multiple jitter cycles for temporal comparisons and match sharpening/filtering. A pipeline trace plus correct-looking screenshot is a smoke test, not a universal correctness proof.

Evidence: Recommended criteria from the observed failure modes and existing data contracts. The 0.01-pixel threshold is a proposed controlled-fixture gate, not a measured guarantee for independently moving translucent layers.

Acceptance matrix: lifecycle and performance

Category	Release gate
Renderer / device	No Sodium; stable Sodium; qualified beta. NVIDIA/AMD/Intel where applicable; Vulkan multidraw and indirect fallback. Unsupported native features report a reason.
Lifecycle	World load/unload, dimension and perspective change, FOV/bob/hurt/nausea, resource-pack reload, graphics options, AA/model/quality switching, pause/resume, resize/minimize/Alt-Tab and direct exit.
Streaming	Fast travel, camera cuts, new section uploads, chunk fade, rebuilt blocks, visible-only animated texture reactivation and entity culling toggles. Avoid global history resets on every harmless upload.
GPU correctness	Core plus synchronization validation; no leaked objects on teardown, illegal descriptor/aspect/layout use or double presentation. Attribute upstream SDK/renderer errors separately.
Performance	CPU profiles with/without object capture at equal entity visibility; GPU pass timings; upload bytes, allocations and VRAM. Warmup, repeated runs and uncapped comparisons outside screenshot capture.
FG evidence	Confirm generated frames and displayed/base FPS separately, frame pacing/latency, low-base-FPS behavior and UI/hand artifact checks. Do not equate a successful swapchain call with complete support.
User state	Normal settings preserved during tests; optional features do not overwrite Sodium preferences silently. Reopen, reset and restart maintain consistent values.

This investigation preserved the normal runClient config/options and kept all test mods/world changes in an isolated profile. Sodium also generated a process-relative mixin-properties file; it was moved into the ignored audit workspace afterward. Production source remained unchanged.

Recommended implementation direction
Build on the integration that already works. Fix the silent MIP mismatch and intrinsic-writer fallback first, repair the independently reproduced FG baseline, then qualify the full matrix. Keep Iris/shader-pack support as a separately researched renderer contract.

Reproducibility and artifact identities

Artifact	Identity
Mod source commit	21b0de753ee11ad936159042887bba12538e7b5b
Mod JAR SHA-256	67f3bde273b7cd5b244c21798671df278d4e55023b0f72dcbb438ab631347377
Sodium stable source	03198cb36ab831c99694403d18e4cf4a1726df72
Sodium beta source	d553ceeea0f33a59aff9258711ae044fdee5e489
Stable runtime SHA-256	87a5bb39f7e6e41106e77f122f82fc93f9d3bc0b6ebcb746b314172a446a287a
Beta runtime SHA-256	84cdb90d273aadbc00cddbcc82ede6e44fd33ac635f23c1d43fec28cc43b9bd6

Runtime files were downloaded from the official Modrinth project metadata and verified against their published SHA-512 values before execution. Exact metadata and source tag pins are included with the evidence. Full SDKs and third-party runtime binaries are not redistributed with this report.

Evidence locations

Compact evidence accompanies the PDF under sodium-compatibility-evidence-2026-10-25. It includes analysis results, validator/launch logs, capture metadata, source-reference index, runtime checksums and representative images. Full raw GPU buffers, cloned sources and disposable scripts remain in build/sodium-compat-audit-20261025 and run/sodium-compat-audit-20261025 in this workspace.

The no-Sodium baseline completed its non-FG cases before the FG crash. Later stable and beta Sodium runs deliberately excluded FG and exited without Vulkan VUID/synchronization errors. The vertex-writer warning remained, as expected. Eight-frame mode captures and 24-frame dynamic samples must not be described as long-duration or all-hardware certification.

Evidence: analysis.json, runtime-identities.json, source-references.json and capture summaries. Tests are traceable by unique labels; failed FG launch logs are retained separately from successful Sodium runs.

Source index

Links target the audited source commit rather than a moving development branch. Local logs and capture JSON provide the measured evidence; proposed changes are identified separately.

[S1] Stable release / 26.3 scope

[S2] Beta release and upstream changes

[S3] Official runtime version metadata

[S4] World renderer replacement hook

[S5] Terrain pass reuse / SodiumChunkSection

[S6] SodiumWorldRenderer

[S7] Chunk worker scheduling

[S8] RenderSectionManager

[S9] Region layout and resource ownership

[S10] Compact terrain vertex format

[S11] DefaultChunkRenderer and pipeline binding

[S12] Backend selection

[S13] Vulkan draw context

[S14] Projection capture / globals upload

[S14b] UniformBufferManager

[S15] Terrain pipeline construction

Source index (continued)

Links target the audited source commit rather than a moving development branch. Local logs and capture JSON provide the measured evidence; proposed changes are identified separately.

[S16] Terrain fragment sampling

[S17] Terrain vertex shader

[S18] Optimized cuboid emission hook

[S19] Vertex-writer conversion and fallback

[S20] Public VertexBufferWriter API

[S21] Chunk fade / fog behavior

[S22] Optimized billboard emission

[S23] Configuration entrypoint registration

[M1] Build dependencies and runtime scope

[M2] Scene target, projection and hand hooks

[M3] World post-effect coordination

[M4] Frontend pipeline variant selection

[M5] MSAA pipeline metadata and state

[M6] MSAA scene/depth lifecycle

[M7] Current terrain MIP source selector

[M8] Texture sampling patch policy

Source index (continued)

Links target the audited source commit rather than a moving development branch. Local logs and capture JSON provide the measured evidence; proposed changes are identified separately.

[M9] Current object capture wrapper

[M10] SSAA/spatial upscale target handling

[M11] Temporal matrices, jitter and resets

[M12] DLSS scene/native boundary

[M13] Opaque/OIT capture hooks

[M13b] FSR scene/native boundary

[M14] Device feature negotiation

[M14b] FG surface/presentation integration

[M15] Existing Sodium configuration adapter